

Seurat QC for sc-/sn-RNA-Seq Analysis in 10X Genomics data

Antonia Chroni for SJCRH DNB_BINF_Core

Contents

1 Information about this notebook **3**

2 Set up **3**

3 Directories and paths to file Inputs/Outputs **3**

4 Read in 10x data **4**

5 Plot data before filtering **6**

6 QC steps for filtering low quality cells **10**

7 Number of cells after filtering **11**

8 Number of cells express a specific gene **11**

9 Number of expressed genes in the library **12**

10 Plot data after filtering **13**

11 Process seurat object **17**
11.1 Normalization of the data 17
11.2 Identification of highly variable features (feature selection) 17
11.3 Scaling the data 17
11.4 Perform linear dimensional reduction (PCA) and Run non-linear dimensional reduction (UMAP) 17

12 Save output files **23**

13 Session Info **24**

```
## The following objects are masked _by_ .GlobalEnv:
##
##     metadata_dir, root_dir
```

PI: Michael A. Dyer, PhD
Project: Addendum-3D2D-EWS-dual-genome
Task: QC and filtering for other samples, i.e., CCLF_cells, PDX, 3D
Project Lead(s): Melody Allensworth
Department: Developmental Neurobiology

DNB Bioinformatics Core Analysis Team:

Lead Analyst(s): Antonia Chroni, PhD
Group Lead: Cody A. Ramirez, PhD
Contact E-mail: antonia.chroni@stjude.org[1]
DNB Bioinformatics Core Pipeline: Standard sc-/sn-RNA-Seq Analysis in 10X Genomics data

Date started: 2025-03-06
Date completed: ONGOING
Report generated: 19:36:33 UTC 03/21/2025

Reviewed by: _____ Date: _____

[1] mailto:antonia.chroni@stjude.org

1 Information about this notebook

This script creates Seurat object from the aligned count data and performs QC based on Seurat functions. For more information or updates, please see Seurat[2].

2 Set up

```
suppressPackageStartupMessages({
  library(future)
  library(cowplot)
  library(devtools)
  library(forcats)
  library(GGally)
  library(stringr)
  library(ggpmisc)
  library(ggrepel)
  library(miQC)
  library(flexmix) # to estimate mixtureModel for miQC
  library(scater)
  library(Seurat)
  library(SingleCellExperiment)
  library(irlba) # this solves the issue with RunUMAP code chunk
  library(scooter)
  library(tidyverse)
  library(fs) #file system
  library(RColorBrewer)

  # Evaluate Seurat R expressions asynchronously when possible using future package
  options(future.globals.maxSize = future_globals_value)
  plan(multisession, workers = parallelly::availableCores())})
```

3 Directories and paths to file Inputs/Outputs

```
attach(params)
```

```
## The following objects are masked _by_ .GlobalEnv:
##
##     metadata_dir, root_dir
```

```
analysis_dir <- file.path(root_dir, "analyses", "upstream-analysis")
module_results_dir <- file.path(analysis_dir, paste0("results"))
SoupX_results_dir <- file.path(module_results_dir, paste0("01_SoupX"))

# We will first read in metadata file as we need to define sample_name
metadata_file <- file.path(metadata_dir, "project_metadata.tsv") # metadata input file
palette_file <- file.path(root_dir, "figures", "palettes", "qc_color_palette.tsv")
gradient_palette_file <- file.path(root_dir, "figures", "palettes", "gradient_color_palette

# Read metadata file and define `sample_name`
```

[2] https://satijalab.org/seurat/articles/pbmc3k_tutorial.html

```
project_metadata <- read.csv(metadata_file, sep = "\t", header = TRUE)
sample_name <- unique(as.character(project_metadata$ID))
sample_name <- sort(sample_name, decreasing = FALSE)
print(sample_name)
```

```
## [1] "DYE1472" "DYE2111" "DYE2547" "DYE2548" "DYE2553" "DYE2554" "DYE2557"
```

```
# File path to `plots` directory
plots_dir <- file.path(analysis_dir, "plots")
seurat_qc_plots_dir <- file.path(plots_dir, "02_Seurat_qc")
samples_plots_dir <- file.path(seurat_qc_plots_dir, sample_name[i])

# File path to `results` directory
module_results_dir <- file.path(analysis_dir, paste0("results"))
seurat_results_dir <- file.path(module_results_dir, paste0("02_Seurat_qc"))
results_dir <- file.path(seurat_results_dir, sample_name[i])

source(paste0(root_dir, "/figures/scripts/theme_plot.R"))
source(paste0(analysis_dir, "/util/function-run-QC.R"))
source(paste0(analysis_dir, "/util/function-calculate-qc-metrics.R"))
source(paste0(analysis_dir, "/util/function-process-Seurat.R"))
source(paste0(analysis_dir, "/util/function-create-UMAP.R"))
```

```
# Read color palette
palette_df <- readr::read_tsv(palette_file, guess_max = 100000, show_col_types = FALSE)
gradient_palette_df <- readr::read_tsv(gradient_palette_file, guess_max = 100000, show_col_t
```

4 Read in 10x data

We will first read read counts in the 10x data. There is the option to use the CellRanger or the corrected matrix as generated by SoupX[3]. This is an optional step, so it will be skipped or not as defined in `params`.

```
cat("Beginning to process sample:", sample_name[i], "\n")
```

```
## Beginning to process sample: DYE2547
```

```
if (use_SoupX_filtering == "YES"){
  print_message <- "we will use SoupX corrected matrix"

    sample_SoupX_input_dir <- file.path(SoupX_results_dir, sample_name[i])
    SoupX_file <- c(dir(path = sample_SoupX_input_dir,  pattern = "mtx_corrected_SoupX.rds",

    counts_mat <- readRDS(SoupX_file)
    #seurat_obj <- CreateSeuratObject(counts = counts_mat,
    #                                 assay = "RNA_SoupX")
    seurat_obj <- create_seurat_obj(counts_matrix = counts_mat,
    #                                 assay = "RNA_SoupX")

    # we will leave name of the assay the same here or we need to update `RNA` everywhere
    # we will update the name of the assay in the `04_run_filter_object.Rmd`
```

[3] https://cran.r-project.org/web/packages/SoupX/vignettes/pbmcTutorial.html

```
                                        assay = "RNA")

   } else {
    print_message <- "we will skip usage of the SoupX corrected matrix and use the CellRange

    input_dir <- file.path(data_dir, sample_name[i], "outs") # CellRanger input file dir

    # File path to data with the filtered_feature_bc_matrix
    cat(paste(c("", sprintf("`%s`", list.files(paste(input_dir, "/outs"), full.names = TRUE)
          collapse = "\n- "))
    cat("\n\n")

    # Load counts to a list of matrices
    counts_mat <- load_sample_counts_matrix(path = input_dir,
                                            sample_name = sample_name[i])
    # Create seurat object using gene expression data
    seurat_obj <- create_seurat_obj(counts_matrix = counts_mat[["Gene Expression"]],
                                    assay = "RNA")
}
```

```
## loading counts matrix for sample: DYE2547
```

```
## loading counts matrix dir: /research/dept/dnb/core_operations/Bioinformatics/dyergrp/Chro
rna-seq-snap-Addendum-3D2D-EWS-dual-genome/analyses/cellranger-analysis/results/02_cellrange
```

```
##
##
##  ========== import cell ranger counts matrix ==========
```

```
##
## ========== create seurat object ==========
##
##
##                     input cells: 25956
##                     input genes: 33991
```

```
colnames(seurat_obj@meta.data)[colnames(seurat_obj@meta.data) == "pct_mito"] = "percent.mito
seurat_obj$log10GenesPerUMI <- log10(seurat_obj$nFeature_RNA)/log10(seurat_obj$nCount_RNA)

# Add metadata
cat("Adding metadata to merged_seurat_obj\n")
```

```
## Adding metadata to merged_seurat_obj
```

```
sample_metadata <- rbind(project_metadata[i, ], project_metadata[i, ][rep(1, length(seurat_o
#seurat_obj@meta.data <- cbind(seurat_obj@meta.data, sample_metadata)

# Let's create `ID` identifying variable in the Seurat metadata that also exists in the proj
# to ensure that we attach the correct data to sample
seurat_obj$ID <- seurat_obj$orig.ident
add_data <- left_join(seurat_obj[["ID"]], project_metadata)
```

```
## Joining with `by = join_by(ID)`
```

```
row.names(add_data) <- row.names(seurat_obj[[]])
seurat_obj <- AddMetaData(seurat_obj, metadata = add_data)
saveRDS(seurat_obj, file.path(results_dir, "/", "seurat_obj_raw.rds")) # Save raw Seurat ob

cat("Number of cells before filter for", sample_name[i], ":", "\n")
```

```
## Number of cells before filter for DYE2547 :
```

```
cells_before_filter_num <- print(ncol(seurat_obj)) # Number of cells before filter
```

```
## [1] 25956
```

Here, we will skip usage of the SoupX corrected matrix and use the CellRanger matrix.

5 Plot data before filtering

Next, we will plot distribution of the number of genes, UMI, and percent mitochondrial reads per cell before filtering any low-quality cells.

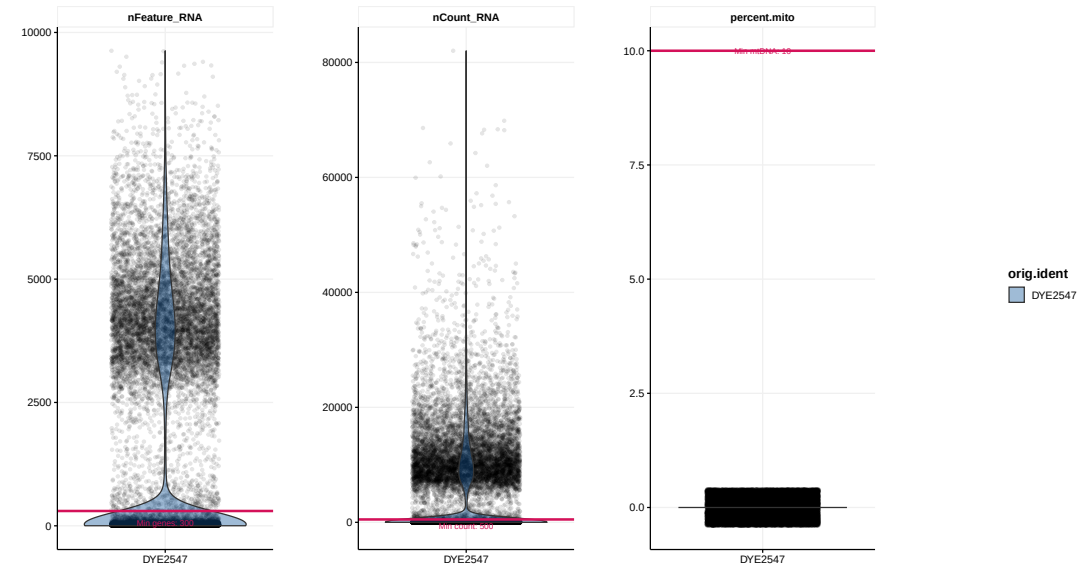
```
cat("Generating QC plots for RAW data for sample", sample_name[i], "\n")
```

```
## Generating QC plots for RAW data for sample DYE2547
```

```
fname <- paste0(samples_plots_dir, "/", sample_name[i], "_", "01", "_before_filter_vln_plot
p <- print(create_vln_plot(seurat_obj = seurat_obj, grouping = grouping, min_genes = min_ge
```

```
## Scale for fill is already present.
## Adding another scale for fill, which will replace the existing scale.
## Scale for fill is already present.
## Adding another scale for fill, which will replace the existing scale.
## Scale for fill is already present.
## Adding another scale for fill, which will replace the existing scale.
```

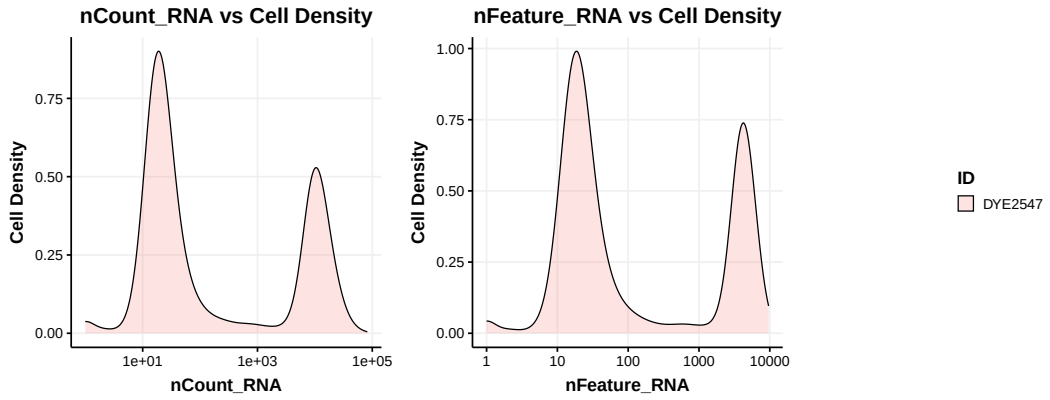
Unfiltered−data



```
ggsave(file = fname, p, width = 8, height = 6, device = "png")

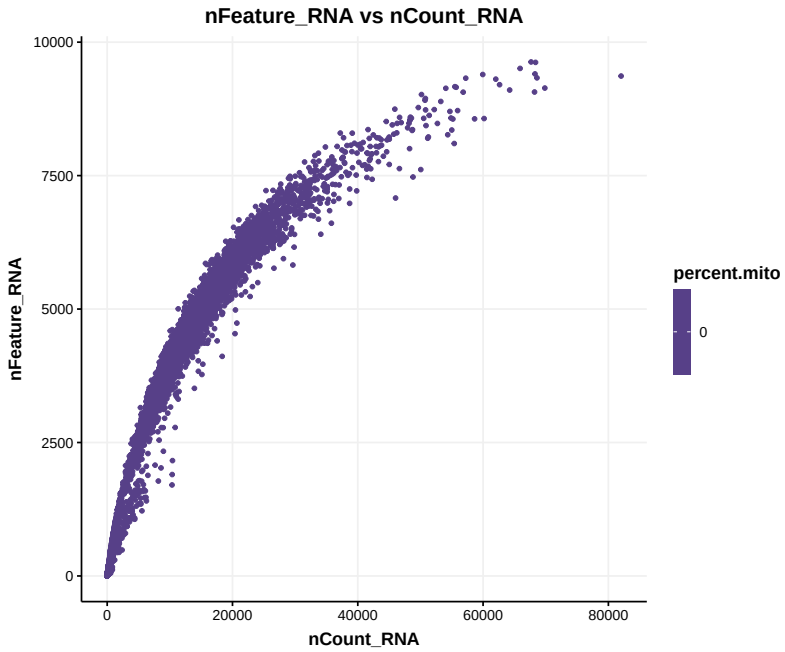
Generate_QC_Plots_1(Seurat_obj = seurat_obj, Project_Path = seurat_qc_plots_dir, Figure_Tota
                    File_Name = "QC_Raw_Data", palette = gradient_palette_df)
```

```
## [1] "/research/dept/dnb/core_operations/Bioinformatics/dyergrp/Chroni_A/Addendum_3D2D_EWS
rna-seq-snap-Addendum-3D2D-EWS-dual-genome/analyses/upstream-analysis/plots/02_Seurat_qc/DYE
```



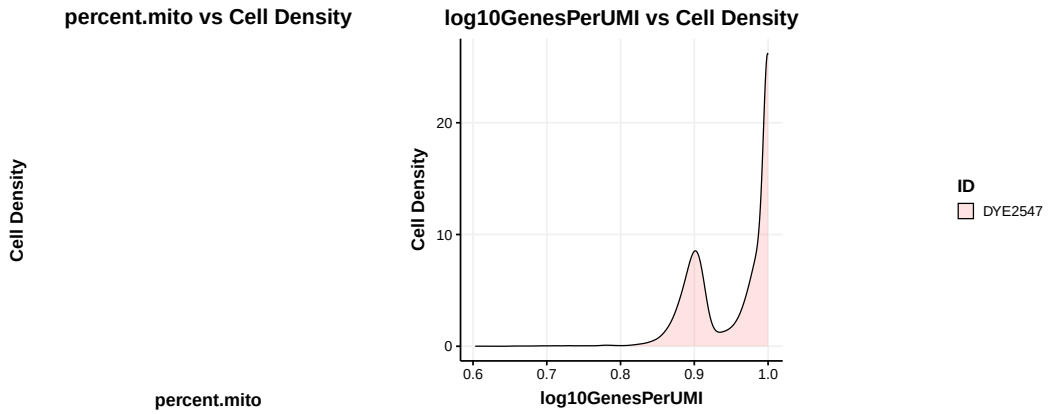

```
Generate_QC_Plots_2(Seurat_obj = seurat_obj, Project_Path = seurat_qc_plots_dir, Figure_Tota
                    File_Name = "QC_Raw_Data", palette = gradient_palette_df)
```

```
## [1] "/research/dept/dnb/core_operations/Bioinformatics/dyergrp/Chroni_A/Addendum_3D2D_EWS
rna-seq-snap-Addendum-3D2D-EWS-dual-genome/analyses/upstream-analysis/plots/02_Seurat_qc/DYE
```



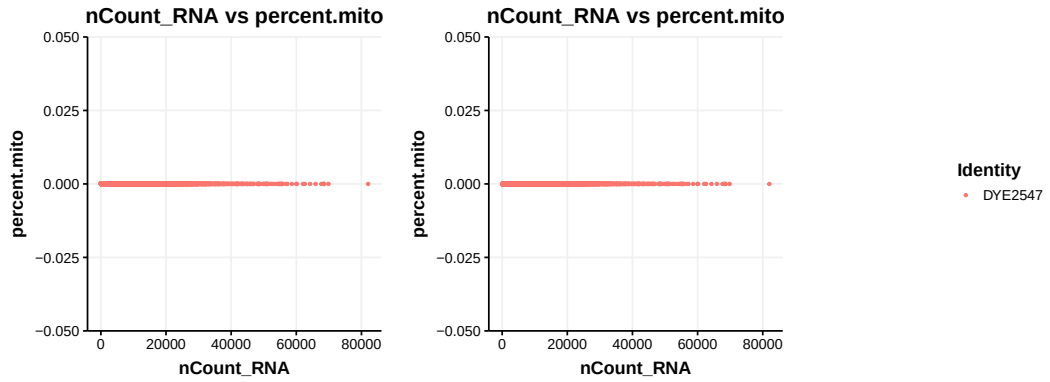

```
Generate_QC_Plots_3(Seurat_obj = seurat_obj, Project_Path = seurat_qc_plots_dir, Figure_Tota
                    File_Name = "QC_Raw_Data", palette = gradient_palette_df)
```

```
## [1] "/research/dept/dnb/core_operations/Bioinformatics/dyergrp/Chroni_A/Addendum_3D2D_EWS
rna-seq-snap-Addendum-3D2D-EWS-dual-genome/analyses/upstream-analysis/plots/02_Seurat_qc/DYE
```



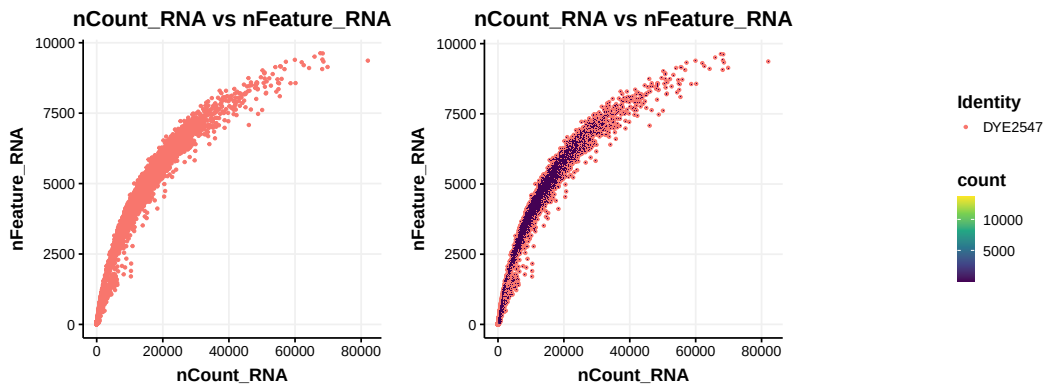

```
Generate_QC_Plots_4(Seurat_obj = seurat_obj, Project_Path = seurat_qc_plots_dir, Figure_Tota
                    File_Name = "QC_Raw_Data", palette = gradient_palette_df)
```

```
## [1] "/research/dept/dnb/core_operations/Bioinformatics/dyergrp/Chroni_A/Addendum_3D2D_EWS
rna-seq-snap-Addendum-3D2D-EWS-dual-genome/analyses/upstream-analysis/plots/02_Seurat_qc/DYE
```



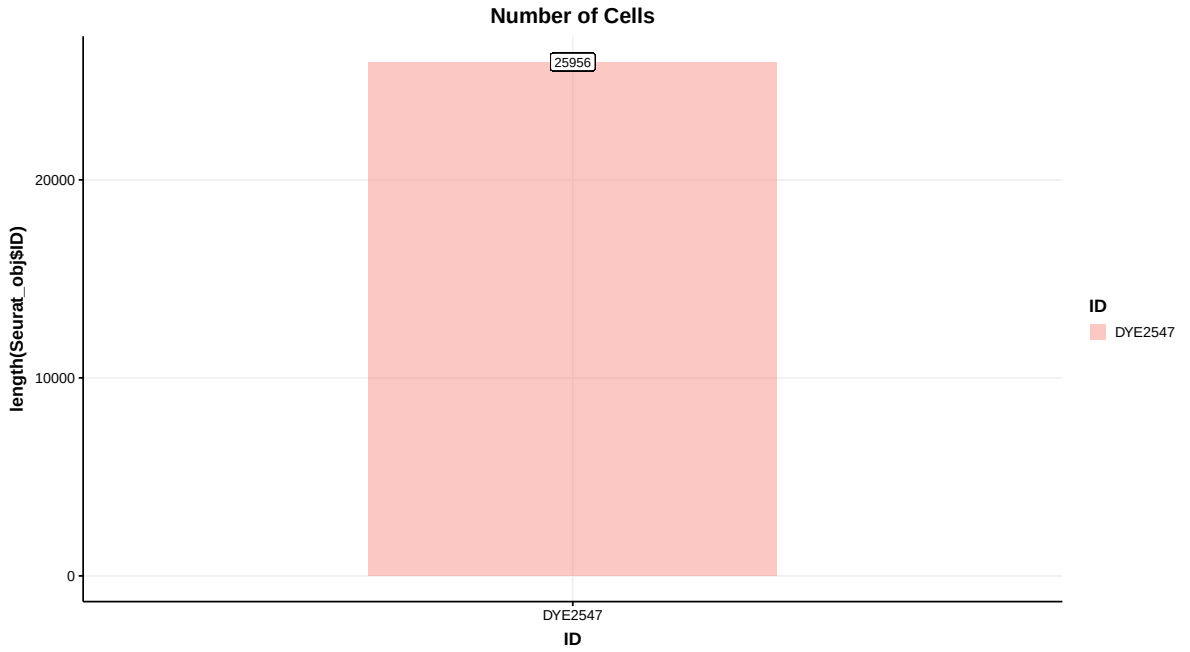

```
Generate_QC_Plots_5(Seurat_obj = seurat_obj, Project_Path = seurat_qc_plots_dir, Figure_Tota
                    File_Name = "QC_Raw_Data", palette = gradient_palette_df)
```

```
## [1] "/research/dept/dnb/core_operations/Bioinformatics/dyergrp/Chroni_A/Addendum_3D2D_EWS
rna-seq-snap-Addendum-3D2D-EWS-dual-genome/analyses/upstream-analysis/plots/02_Seurat_qc/DYE
```




```
Generate_QC_Plots_6(Seurat_obj = seurat_obj, Project_Path = seurat_qc_plots_dir, Figure_Tota
                    File_Name = "QC_Raw_Data", palette = gradient_palette_df)
```

```
## [1] "/research/dept/dnb/core_operations/Bioinformatics/dyergrp/Chroni_A/Addendum_3D2D_EWS
rna-seq-snap-Addendum-3D2D-EWS-dual-genome/analyses/upstream-analysis/plots/02_Seurat_qc/DYE
```



6 QC steps for filtering low quality cells

First, we will use miQC[4] R package to identify low-quality cells. miQC R package jointly models proportion of reads belonging to mitochondrial genes and number of unique genes detected. Cells with a high likelihood of being compromised (greater than 0.75) and cells that do not pass a minimum number of unique genes detected threshold of 200 will be removed from the counts matrix object. The cells at the very top of the graph are almost certainly compromised, most likely to have been derived from the distribution with fewer unique genes and higher baseline mitochondrial expression. In order to calculate the percent of reads in a cell that map to mitochondrial genes, we first need to establish which genes are mitochondrial. For genes listed as HGNC symbols, this is as simple as searching for genes starting with mt-. For other IDs, we recommend using a biomaRt query to map to chromosomal location and identify all mitochondrial genes. miQC is designed to be run with the Bioconductor package scater, which has a built-in function addPerCellQC to calculate basic QC metrics like number of unique genes detected per cell and total number of reads. When we pass in our list of mitochondrial genes, it will also calculate percent mitochondrial reads.

If miQC does not identify any low quality cells, then we will use the `mtDNA_pct_default` value to remove mtDNA from each library (as defined in the `params`). Then, we will use `min_genes` value to remove cells with loc content of genes expressed from each library (as defined in the `params`). Lastly, if defined by the user, we will apply the `Find_Outlier_Thershold` function as a final filtering step (as defined in the `params`).

```
cat("Run miQC for", sample_name[i], "\n")
```

```
## Run miQC for DYE2547
```

```
sce <- as.SingleCellExperiment(seurat_obj) # Create sce

# Indicates whether or not to use miQC
if (use_miQC == "YES"){
```

[4] https://bioconductor.org/packages/devel/bioc/vignettes/miQC/inst/doc/miQC.html

```
  print("We will use miQC.")
  set.seed(1234) # Set seed
  mt_genes <- grepl(sce$percent.mito, rownames(sce)) # Identify cells with mtDNA in the lib
  feature_ctrls <- list(mito = rownames(sce)[mt_genes])
  sce <- addPerCellQC(sce, subsets = feature_ctrls)
  #head(colData(sce))
  print(plotMetrics(sce))
  seurat_obj <- run_miQC(sce) # Run QC function
  } else {
    print("We will QC with default parameters and/or the `Find_Outlier_Thershold` function."
    seurat_obj <- run_QC_default(seurat_obj = seurat_obj)}
```

```
## [1] "We will QC with default parameters and/or the `Find_Outlier_Thershold` function."
## [1] "Use step 1 with default parameters for filtering and then use the `Find_Outlier_Ther
## Step 1 data filter DYE2547
## Step 2 data filter DYE2547
```

7 Number of cells after filtering

```
cat("Number of cells after filtering for sample", sample_name[i], ":", "\n")
```

```
## Number of cells after filtering for sample DYE2547 :
```

```
cells_so_after_filter_num <- print(ncol(seurat_obj)) # Number of cells after filter
```

```
## [1] 8370
```

8 Number of cells express a specific gene

We will calculate percentage of cells expressing each gene and their mean.

```
cat("Number of cells express a specific gene for sample", sample_name[i], "\n")
```

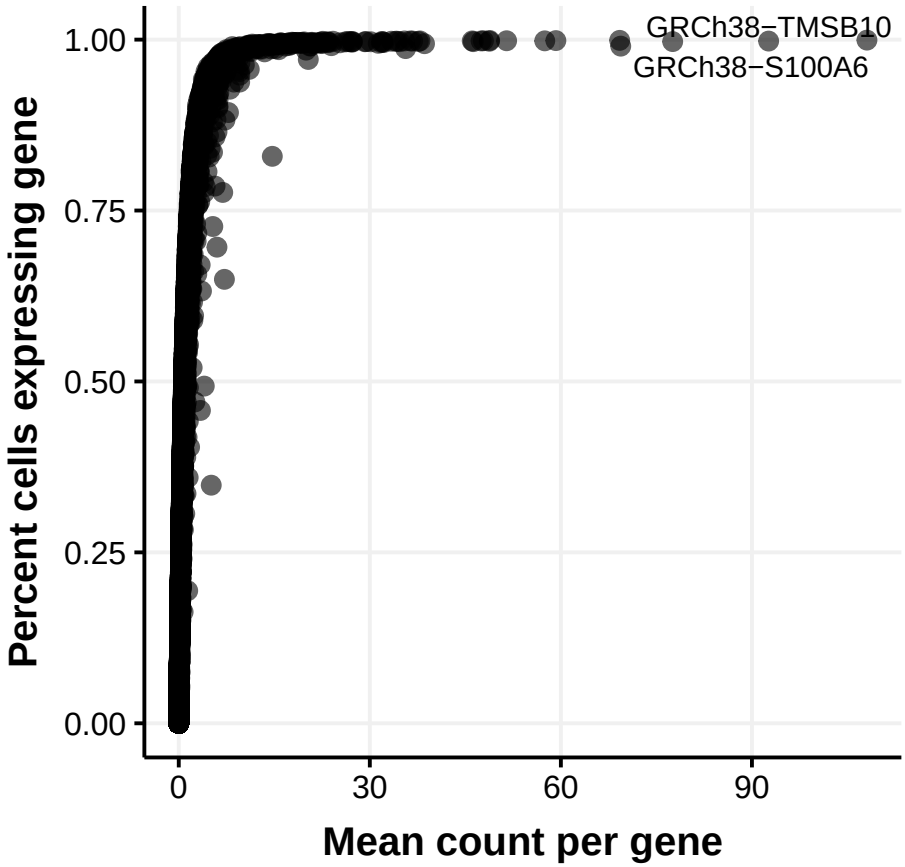
```
## Number of cells express a specific gene for sample DYE2547
```

```
pct_cells <- apply(as.data.frame(seurat_obj@assays$RNA@counts), 1, function(x) (sum(x != 0)
gene_means <- rowMeans(as.data.frame(seurat_obj@assays$RNA@counts))
p <- as.data.frame(cbind(pct_cells, gene_means)) %>%
  rownames_to_column("gene")

fname <- paste0(samples_plots_dir, "/", sample_name[i], "_", "08", "_cells_num_per_gene.png"
print(fname)
```

```
## [1] "/research/dept/dnb/core_operations/Bioinformatics/dyergrp/Chroni_A/Addendum_3D2D_EWS
rna-seq-snap-Addendum-3D2D-EWS-dual-genome/analyses/upstream-analysis/plots/02_Seurat_qc/DYE
```

```
set.seed(1234) # Set seed
plot <- print(ggplot(data = p, aes(x = gene_means, y = pct_cells, label = gene)) +
        theme_Publication () +
        geom_point(size = 3, alpha = 0.6) +
        xlab("Mean count per gene") +
        ylab("Percent cells expressing gene") +
        stat_dens2d_labels(geom = "text_repel", keep.fraction = 0.001))
```




```
ggsave(filename = fname, plot, width = 5, height = 5, device = "png") # Save the plot
```

9 Number of expressed genes in the library

We will calculate the number of expressed genes in the library.

```
cat("Number of expressed genes in the library for sample", sample_name[i], "\n")
```

```
## Number of expressed genes in the library for sample DYE2547
```

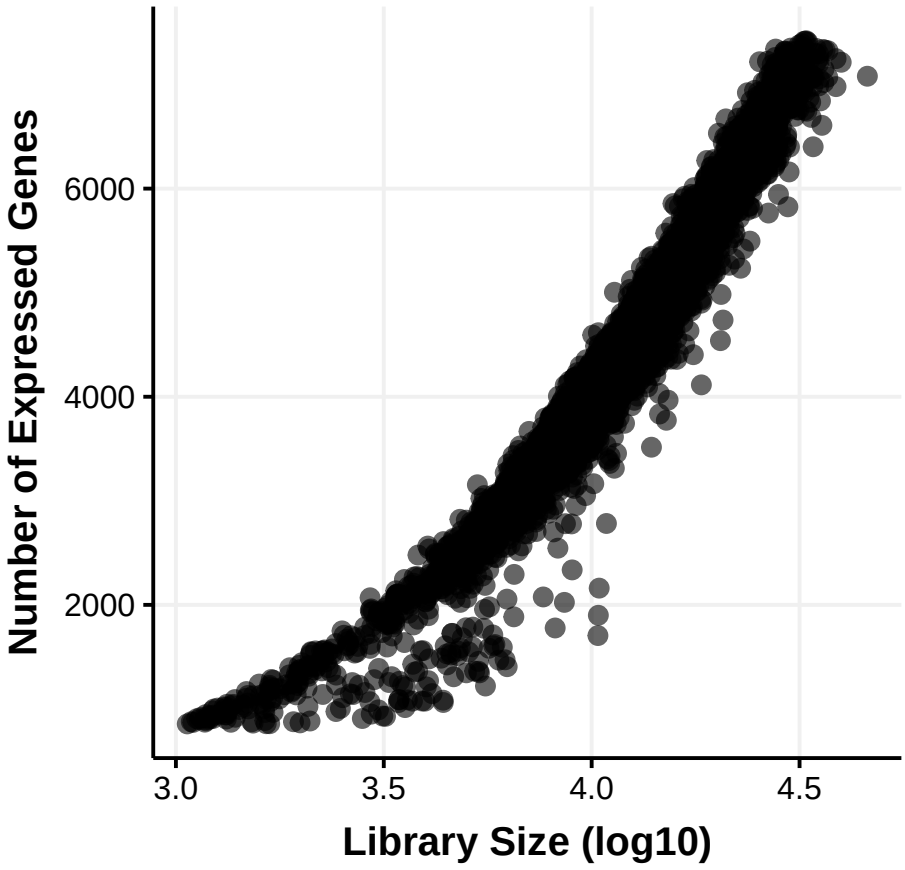
```
num_genes <- apply(as.data.frame(seurat_obj@assays$RNA@counts), 2, function(x) (sum(x != 0))
libsize <- colSums(as.data.frame(seurat_obj@assays$RNA@counts))
l  <- as.data.frame(cbind(num_genes, libsize))

fname <- paste0(samples_plots_dir, "/", sample_name[i], "_", "09", "_genes_num_library.png")
print(fname)
```

```
## [1] "/research/dept/dnb/core_operations/Bioinformatics/dyergrp/Chroni_A/Addendum_3D2D_EWS
rna-seq-snap-Addendum-3D2D-EWS-dual-genome/analyses/upstream-analysis/plots/02_Seurat_qc/DYE
```

```
set.seed(1234) # Set seed
plot <- print(ggplot(data = l, aes(x = log10(libsize), y = num_genes)) +
                theme_Publication () +
                geom_point(size = 3, alpha = 0.6) +
                xlab("Library Size (log10)") +
```

```
                ylab("Number of Expressed Genes"))
```




```
ggsave(filename = fname, plot, width = 5, height = 5, device = "png") # Save the plot
```

10 Plot data after filtering

Next, we will plot distribution of the number of genes, UMI, and percent mitochondrial reads per cell after filtering any low-quality cells.

```
cat("Generating QC plots for FILTERED data for sample", sample_name[i], "\n")
```

```
## Generating QC plots for FILTERED data for sample DYE2547
```

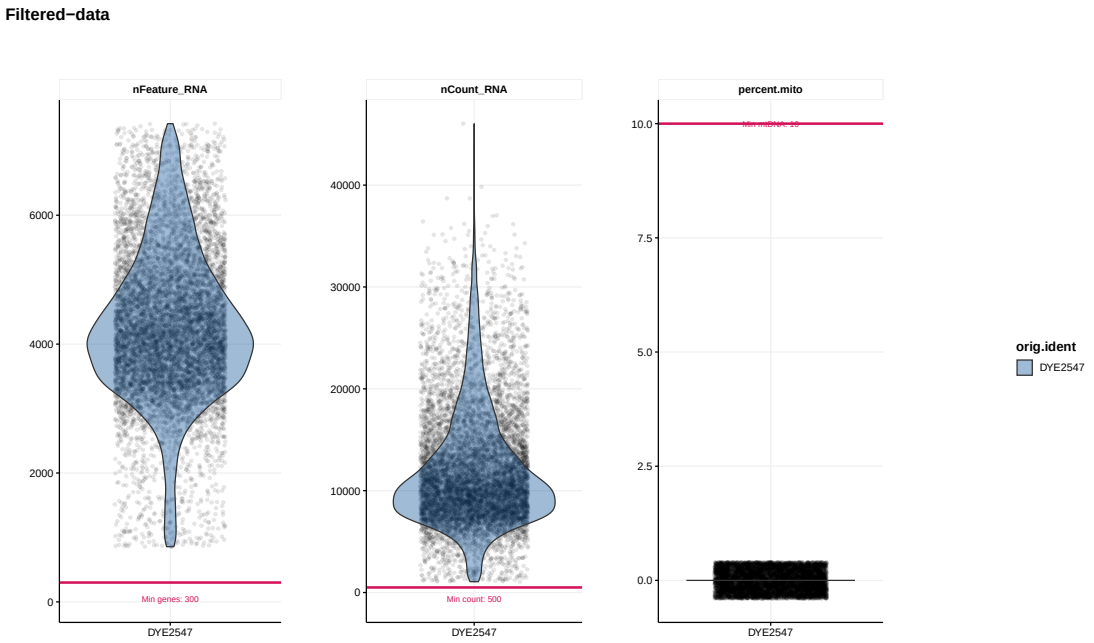
```
fname <- paste0(samples_plots_dir, "/", sample_name[i], "_", "10", "_after_filter_vln_plot.p
print(fname)
```

```
## [1] "/research/dept/dnb/core_operations/Bioinformatics/dyergrp/Chroni_A/Addendum_3D2D_EWS
rna-seq-snap-Addendum-3D2D-EWS-dual-genome/analyses/upstream-analysis/plots/02_Seurat_qc/DYE
```

```
p <- print(create_vln_plot(seurat_obj = seurat_obj, grouping = grouping, min_genes = min_ge
```

```
## Scale for fill is already present.
## Adding another scale for fill, which will replace the existing scale.
## Scale for fill is already present.
## Adding another scale for fill, which will replace the existing scale.
## Scale for fill is already present.
```

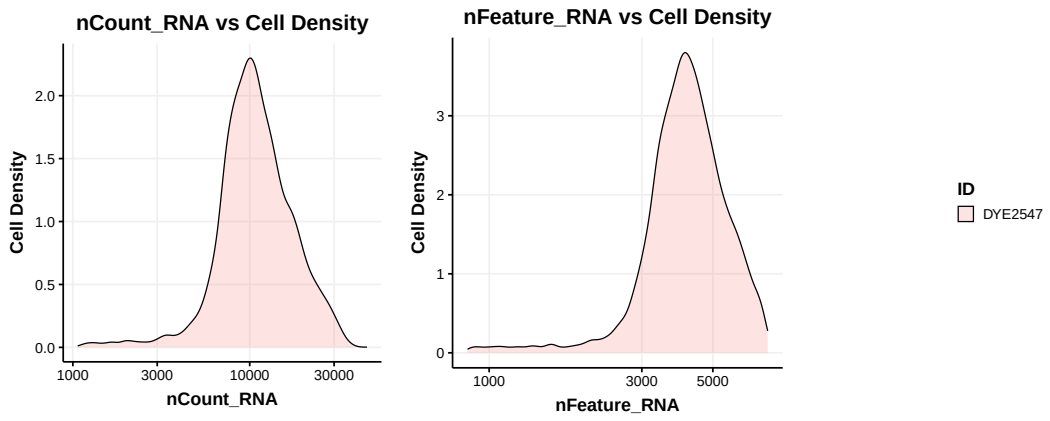
```
## Adding another scale for fill, which will replace the existing scale.
```




```
ggsave(file = fname, p, width = 8, height = 6, device = "png")

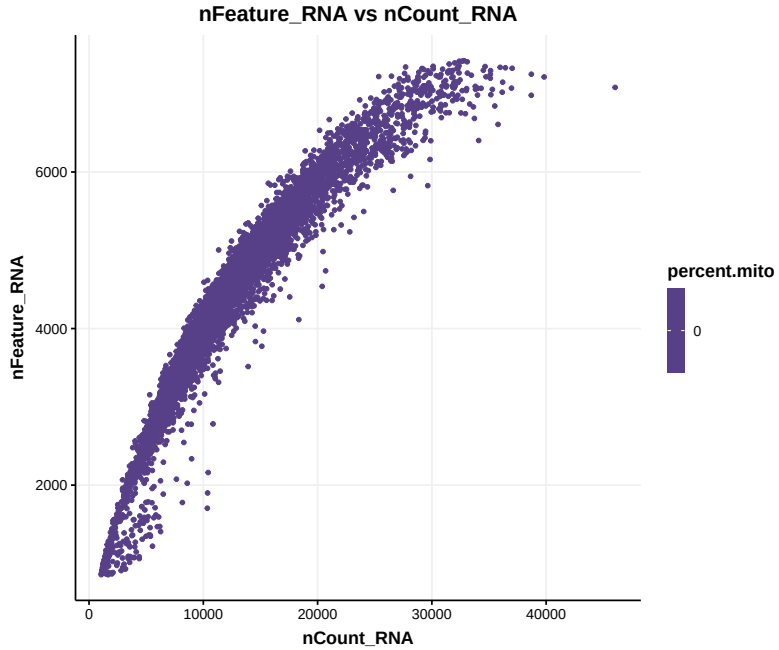
Generate_QC_Plots_1(Seurat_obj = seurat_obj, Project_Path = seurat_qc_plots_dir, Figure_Tota
                    File_Name = "QC_Filtered_Data", palette = gradient_palette_df)
```

```
## [1] "/research/dept/dnb/core_operations/Bioinformatics/dyergrp/Chroni_A/Addendum_3D2D_EWS
rna-seq-snap-Addendum-3D2D-EWS-dual-genome/analyses/upstream-analysis/plots/02_Seurat_qc/DYE
```



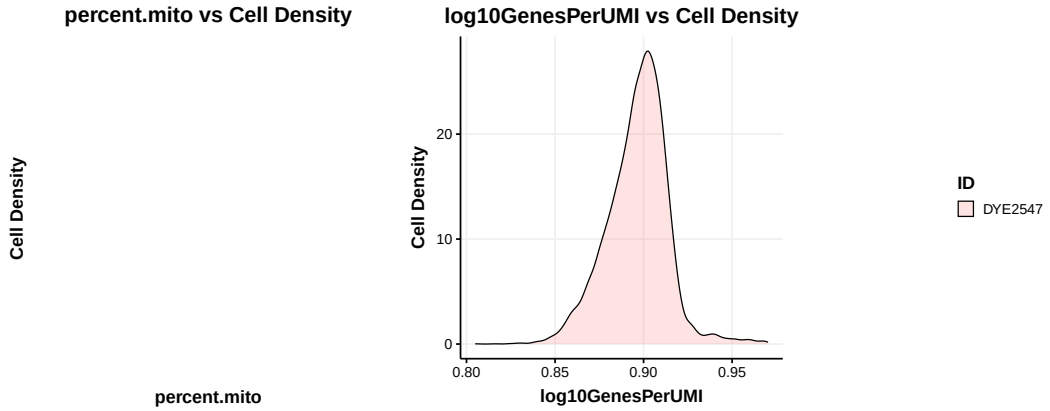

```
Generate_QC_Plots_2(Seurat_obj = seurat_obj, Project_Path = seurat_qc_plots_dir, Figure_Tota
                    File_Name = "QC_Filtered_Data", palette = gradient_palette_df)
```

```
## [1] "/research/dept/dnb/core_operations/Bioinformatics/dyergrp/Chroni_A/Addendum_3D2D_EWS
rna-seq-snap-Addendum-3D2D-EWS-dual-genome/analyses/upstream-analysis/plots/02_Seurat_qc/DYE
```




```
Generate_QC_Plots_3(Seurat_obj = seurat_obj, Project_Path = seurat_qc_plots_dir, Figure_Tota
                    File_Name = "QC_Filtered_Data", palette = gradient_palette_df)
```

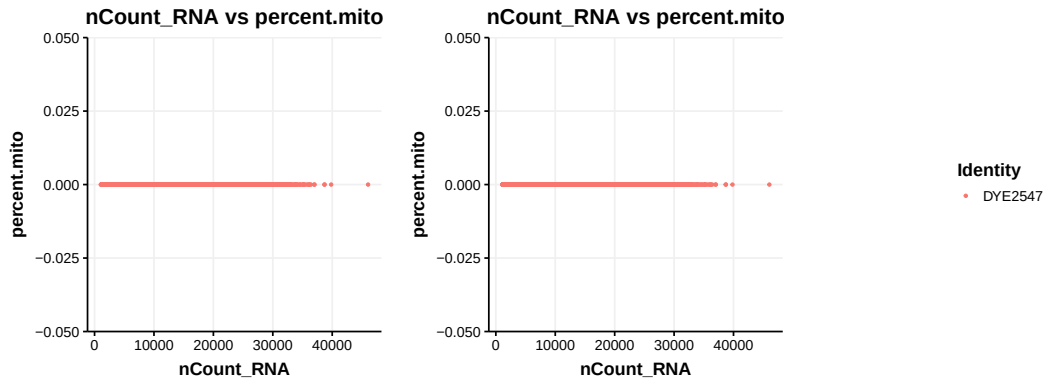
```
## [1] "/research/dept/dnb/core_operations/Bioinformatics/dyergrp/Chroni_A/Addendum_3D2D_EWS
rna-seq-snap-Addendum-3D2D-EWS-dual-genome/analyses/upstream-analysis/plots/02_Seurat_qc/DYE
```




```
Generate_QC_Plots_4(Seurat_obj = seurat_obj, Project_Path = seurat_qc_plots_dir, Figure_Tota
                    File_Name = "QC_Filtered_Data", palette = gradient_palette_df)
```

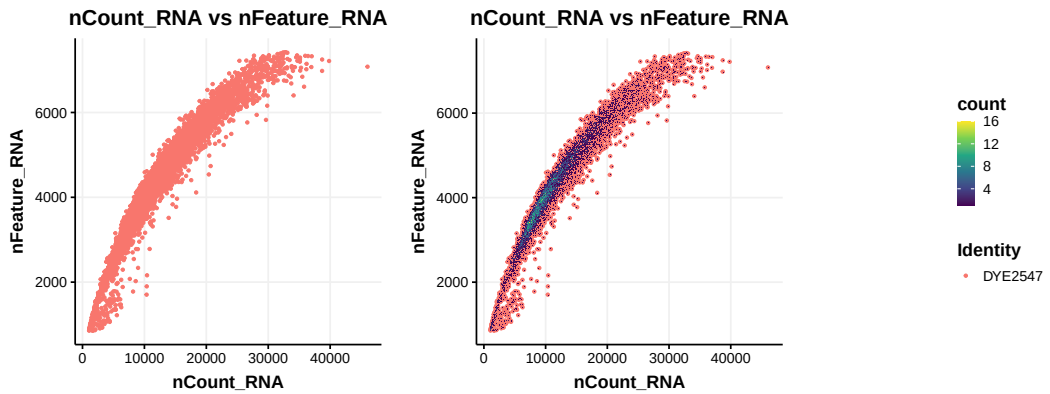
```
## [1] "/research/dept/dnb/core_operations/Bioinformatics/dyergrp/Chroni_A/Addendum_3D2D_EWS
```

```
rna-seq-snap-Addendum-3D2D-EWS-dual-genome/analyses/upstream-analysis/plots/02_Seurat_qc/DYE
```



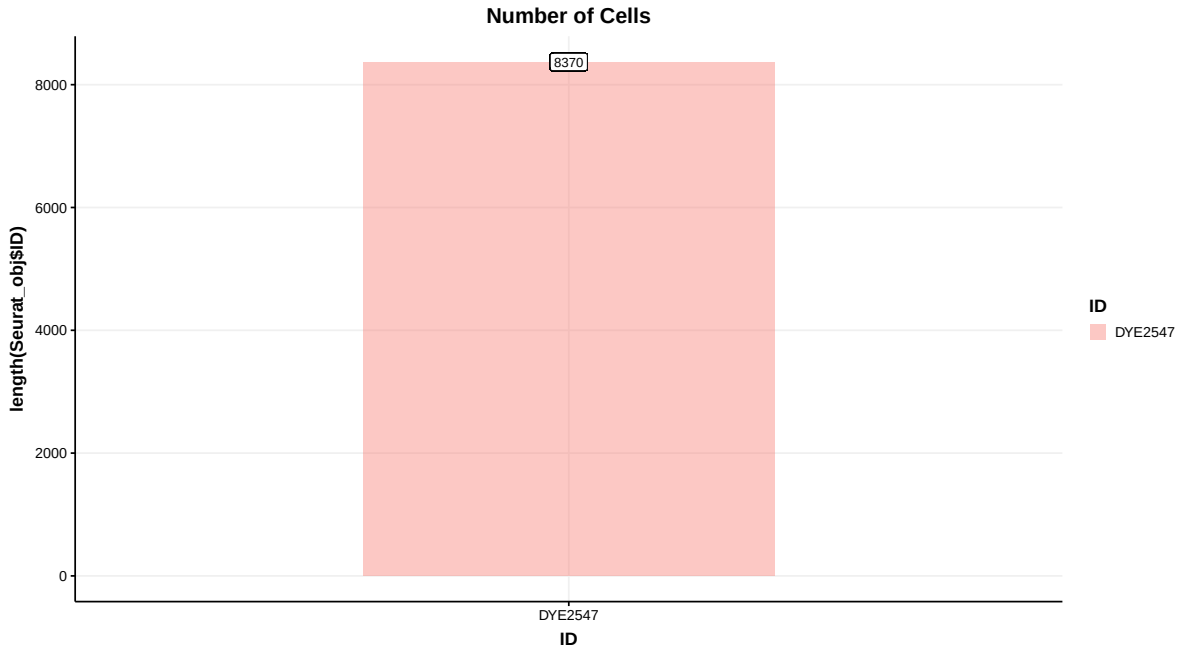

```
Generate_QC_Plots_5(Seurat_obj = seurat_obj, Project_Path = seurat_qc_plots_dir, Figure_Tota
                    File_Name = "QC_Filtered_Data", palette = gradient_palette_df)
```

```
## [1] "/research/dept/dnb/core_operations/Bioinformatics/dyergrp/Chroni_A/Addendum_3D2D_EWS
rna-seq-snap-Addendum-3D2D-EWS-dual-genome/analyses/upstream-analysis/plots/02_Seurat_qc/DYE
```




```
Generate_QC_Plots_6(Seurat_obj = seurat_obj, Project_Path = seurat_qc_plots_dir, Figure_Tota
                    File_Name = "QC_Filtered_Data", palette = gradient_palette_df)
```

```
## [1] "/research/dept/dnb/core_operations/Bioinformatics/dyergrp/Chroni_A/Addendum_3D2D_EWS
rna-seq-snap-Addendum-3D2D-EWS-dual-genome/analyses/upstream-analysis/plots/02_Seurat_qc/DYE
```



11 Process seurat object

The next steps include:

11.1 Normalization of the data

After removing unwanted cells from the dataset, the next step is to normalize the data. By default, we employ a global-scaling normalization method "LogNormalize" that normalizes the feature expression measurements for each cell by the total expression, multiplies this by a scale factor (10,000 by default), and log-transforms the result.

11.2 Identification of highly variable features (feature selection)

Next, we will calculate a subset of features that exhibit high cell-to-cell variation in the dataset (i.e, they are highly expressed in some cells, and lowly expressed in others). We and others have found that focusing on these genes in downstream analysis helps to highlight biological signal in single-cell datasets. By default, Seurat returns 2,000 features per dataset. These will be used in downstream analysis, like PCA.

11.3 Scaling the data

Next, we apply a linear transformation ('scaling') that is a standard pre-processing step prior to dimensional reduction techniques like PCA.

11.4 Perform linear dimensional reduction (PCA) and Run non-linear dimensional reduction (UMAP)

Next we perform PCA on the scaled data. By default, only the previously determined variable features are used as input, but can be defined using features argument if you wish to choose a different subset (if you do want to use a custom subset of features, make sure you pass these to ScaleData first). For the first principal components, Seurat outputs a list of genes with the most positive and negative loadings, representing modules of genes that exhibit either correlation

(or anti-correlation) across single-cells in the dataset. Next, we will run UMAP to visualize and explore the data. The goal of non-linear dimensional reduction algorithms is to learn underlying structure in the dataset, in order to place similar cells together in low-dimensional space. Therefore, cells that are grouped together within graph-based clusters determined above should co-localize on these dimension reduction plots.

```
cat("Create and process seurat for sample", sample_name[i], "\n")
```

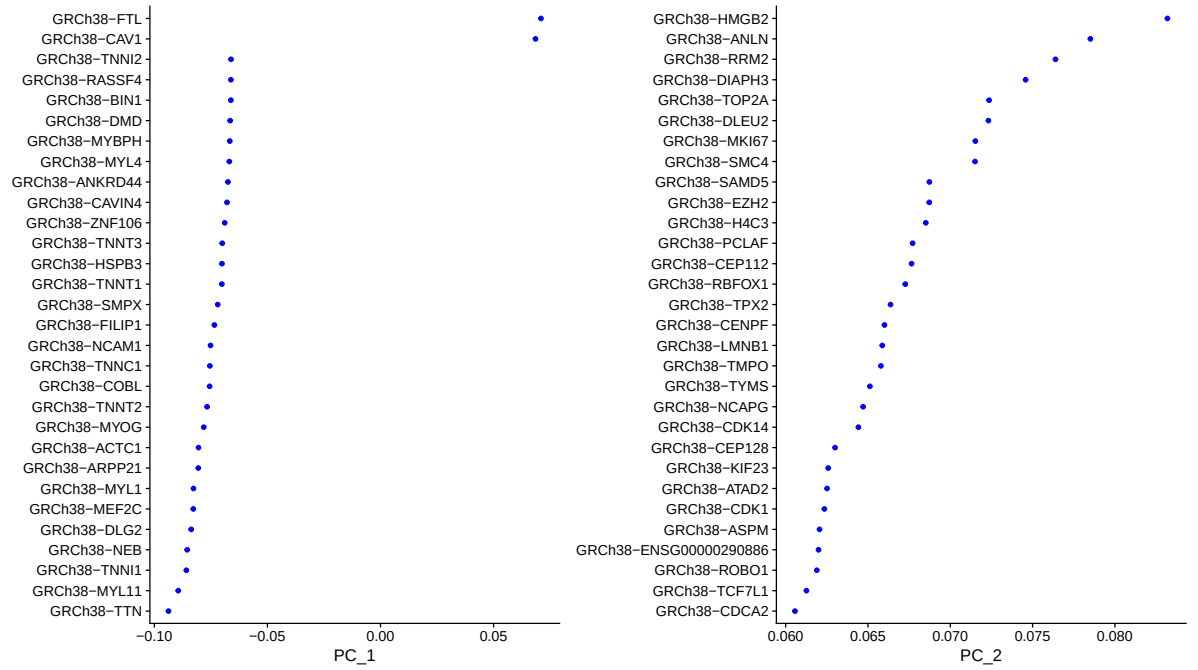
```
## Create and process seurat for sample DYE2547
```

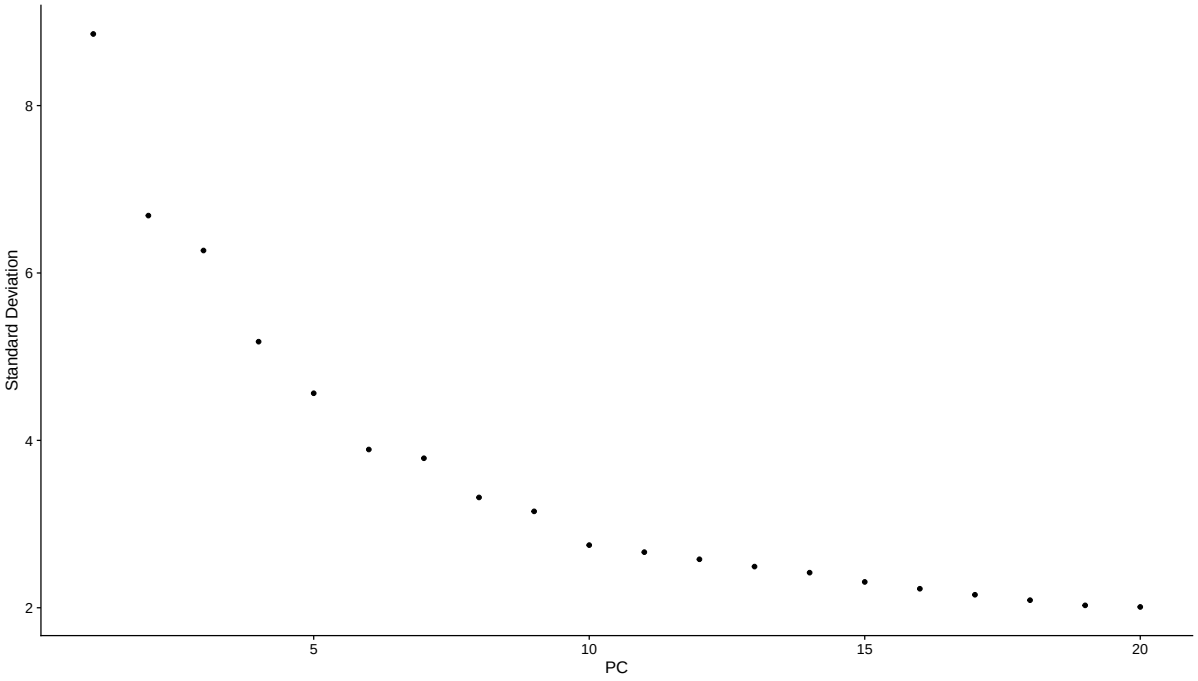
```
seurat_obj <- Process_Seurat(seurat_obj = seurat_obj, nfeatures_value = nfeatures_value, Ge
                             Regress_Cell_Cycle = Regress_Cell_Cycle_value, assay = assay, 
                             prefix = prefix, num_dim = num_dim, num_neighbors = num_neighb
                             plots_output = samples_plots_dir, use_condition_split = use_co
                             condition1 = condition_value1, condition2 = condition_value2, 
                             print_pdf = print_pdf, PCA_Feature_List = PCA_Feature_List_valu
```

```
## Get cell cycle scores and then scale data
## Scale data
## [1] "We will NOT regress out for cell cycle and scale data!"
```

```
## Centering and scaling data matrix
```

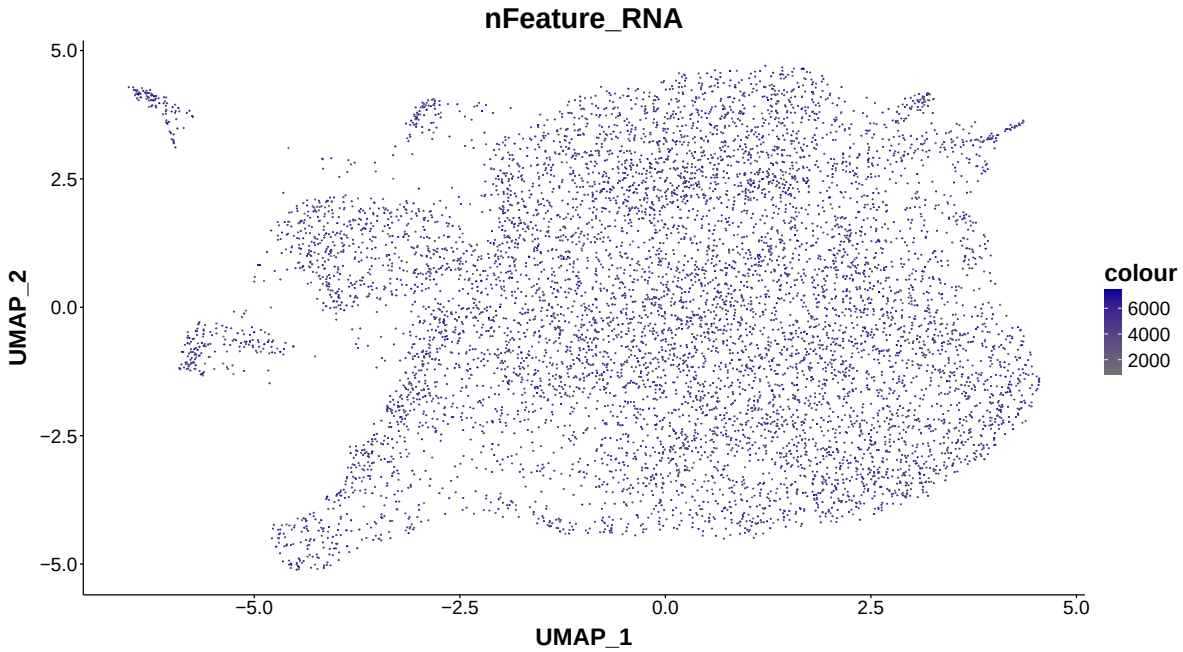
```
## Run PCA
## Run UMAP
## Plot the plots
##  [1] "GRCh38-MYL11" "GRCh38-MYL1"  "GRCh38-TNNC1" "GRCh38-TNNI1" "GRCh38-
SLN"
##  [6] "GRCh38-CXCL8" "GRCh38-IFIT3" "GRCh38-DLG2"  "GRCh38-TTN"   "GRCh38-
TNNT3"
## PC_ 1
## Positive:  GRCh38-FTL, GRCh38-CAV1, GRCh38-MT2A, GRCh38-HMGA2, GRCh38-NRG1
## Negative:  GRCh38-TTN, GRCh38-MYL11, GRCh38-TNNI1, GRCh38-NEB, GRCh38-DLG2
## PC_ 2
## Positive:  GRCh38-HMGB2, GRCh38-ANLN, GRCh38-RRM2, GRCh38-DIAPH3, GRCh38-
TOP2A
## Negative:  GRCh38-RPS28, GRCh38-RPS12, GRCh38-FTH1, GRCh38-RPL36, GRCh38-
SEC61G
## PC_ 3
## Positive:  GRCh38-PRR16, GRCh38-NEAT1, GRCh38-SDK1, GRCh38-PSD3, GRCh38-
ARHGAP24
## Negative:  GRCh38-UBE2S, GRCh38-HMGB2, GRCh38-CKS1B, GRCh38-UBE2C, GRCh38-
CDC20
## PC_ 4
## Positive:  GRCh38-CASQ2, GRCh38-FOSL1, GRCh38-NTM, GRCh38-PLAT, GRCh38-
SMURF2
## Negative:  GRCh38-COL3A1, GRCh38-COL25A1, GRCh38-NCOA1, GRCh38-PDE4D, GRCh38-
OLFML2B
## PC_ 5
## Positive:  GRCh38-ARL6IP1, GRCh38-TOP2A, GRCh38-UBE2C, GRCh38-CENPF, GRCh38-
AURKA
## Negative:  GRCh38-CCNE2, GRCh38-PCNA, GRCh38-POLA1, GRCh38-FAM111B, GRCh38-
MCM6
```

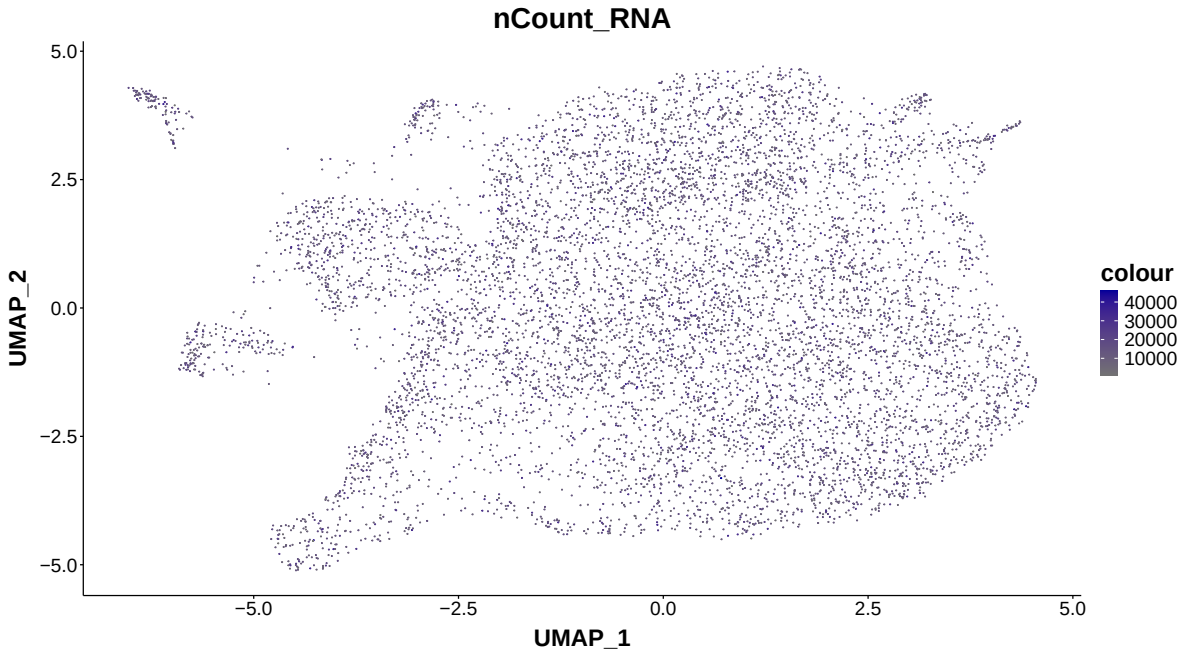


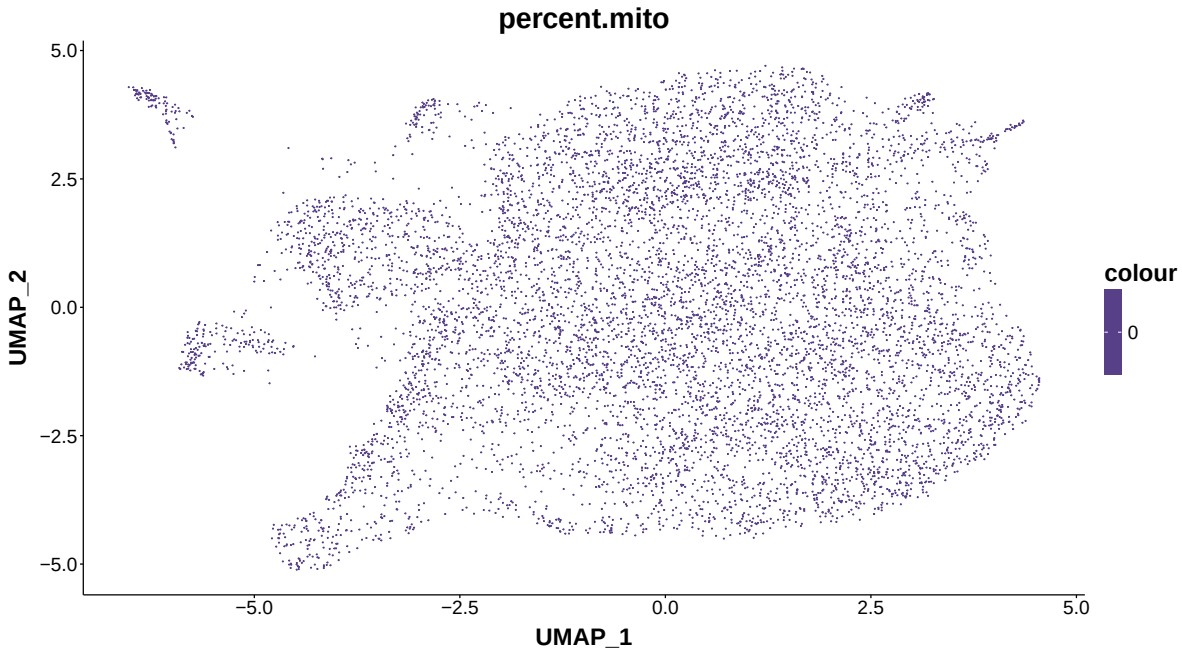


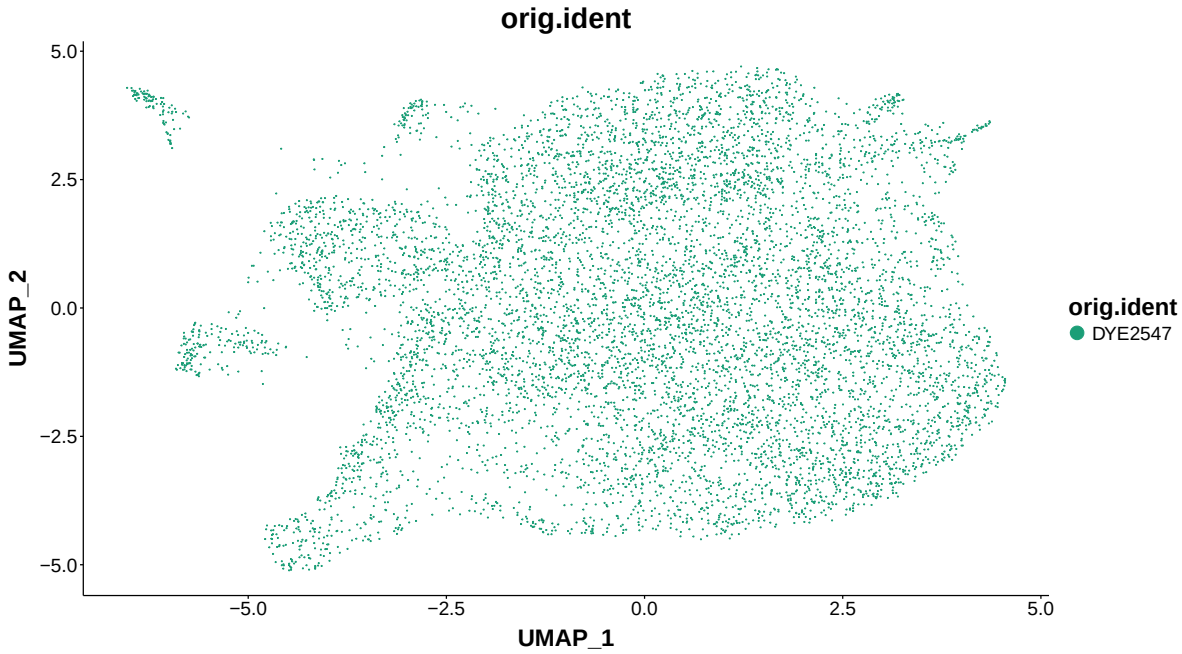

```
## Plot UMAP
## [1] "UMAPndim30nn30"

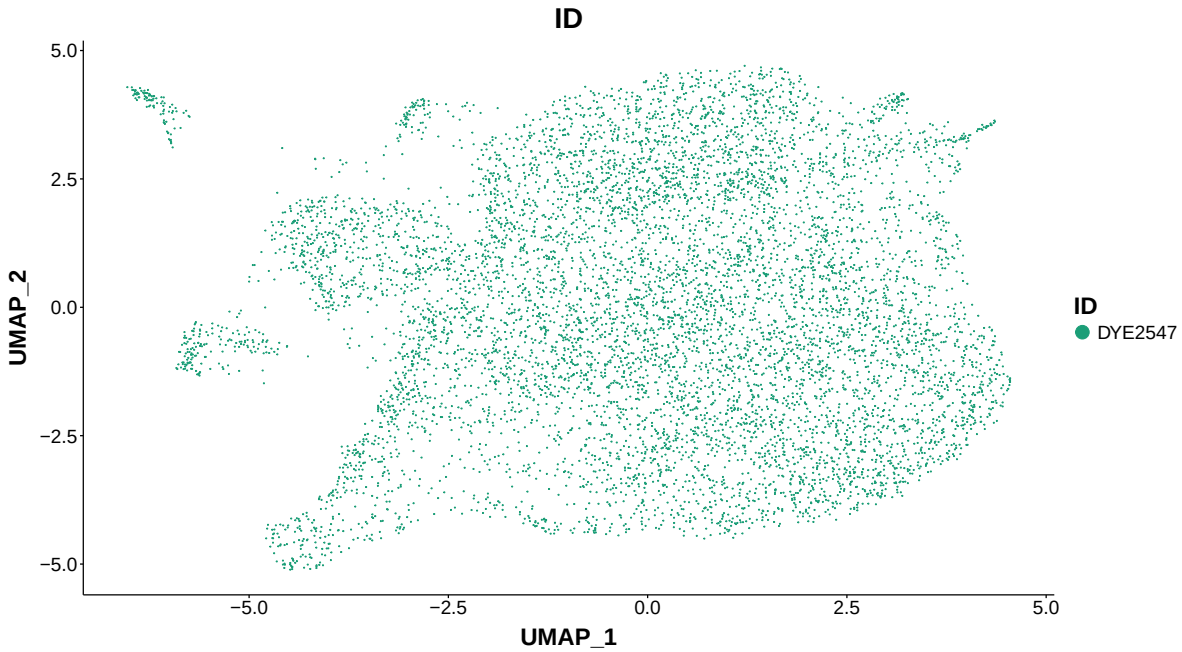
## [1] "/research/dept/dnb/core_operations/Bioinformatics/dyergrp/Chroni_A/Addendum_3D2D_EWS
rna-seq-snap-Addendum-3D2D-EWS-dual-genome/analyses/upstream-analysis/plots/02_Seurat_qc/DYP
```

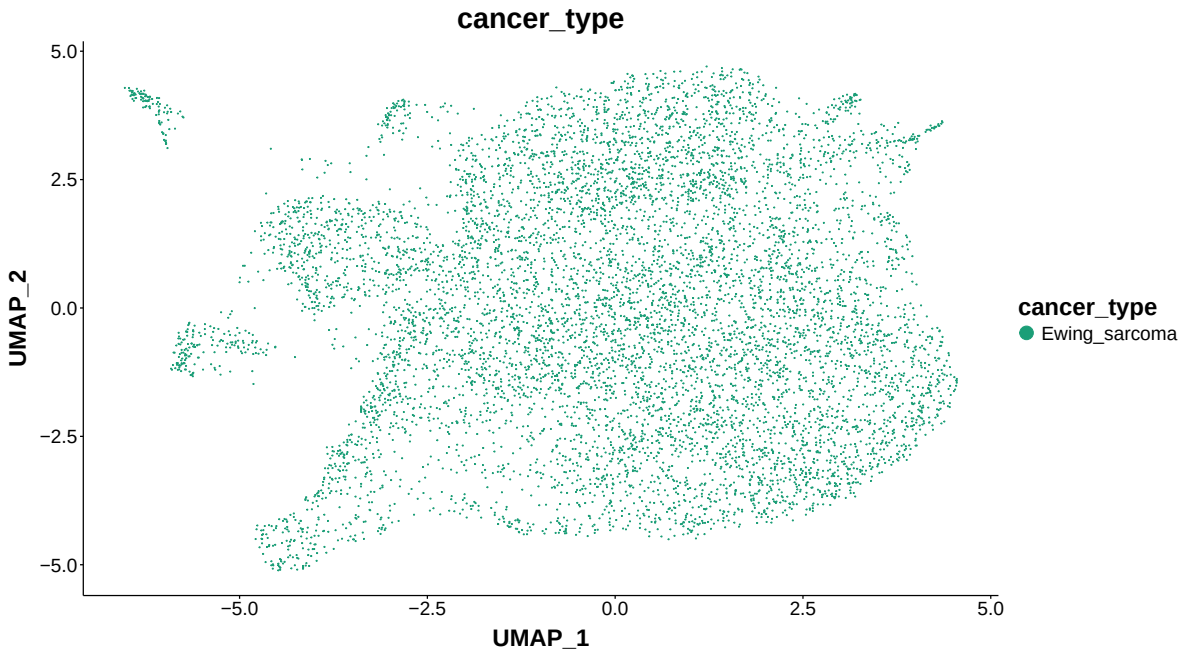

nFeature_RNA
5.0
2.5
0.0
−2.5
−5.0
UMAP_2
−5.0
−2.5
0.0
2.5
5.0
UMAP_1
colour
6000
4000
2000

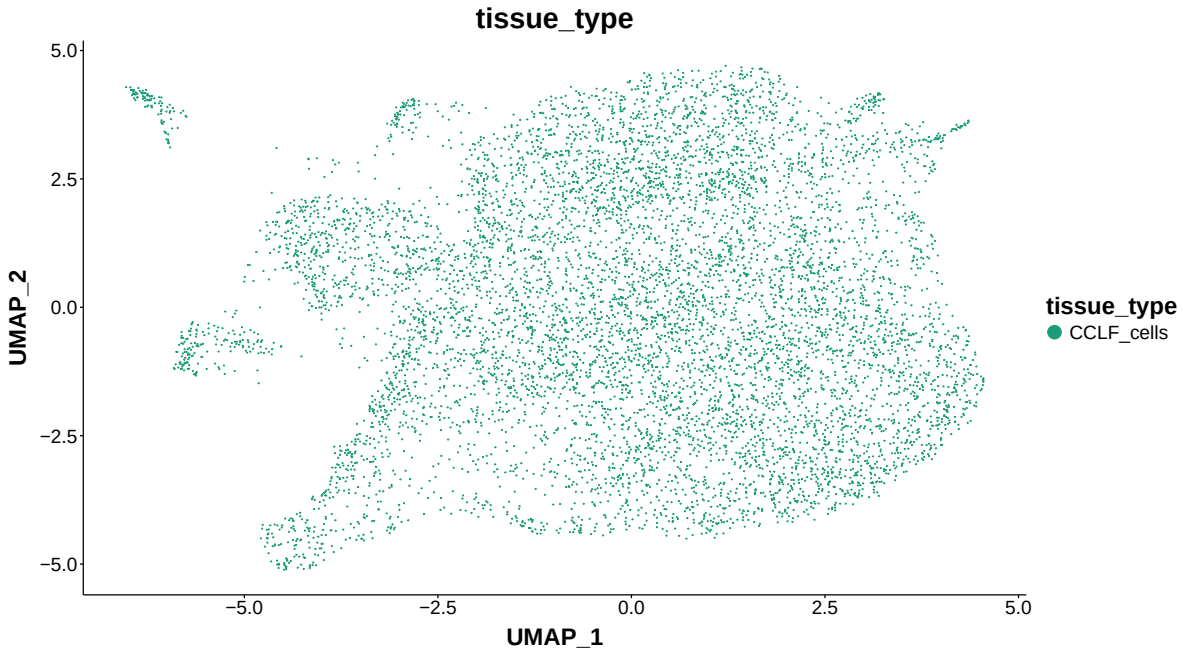

nCount_RNA
5.0
2.5
0.0
−2.5
−5.0
UMAP_2
−5.0
−2.5
0.0
2.5
5.0
UMAP_1
colour
40000
30000
20000
10000

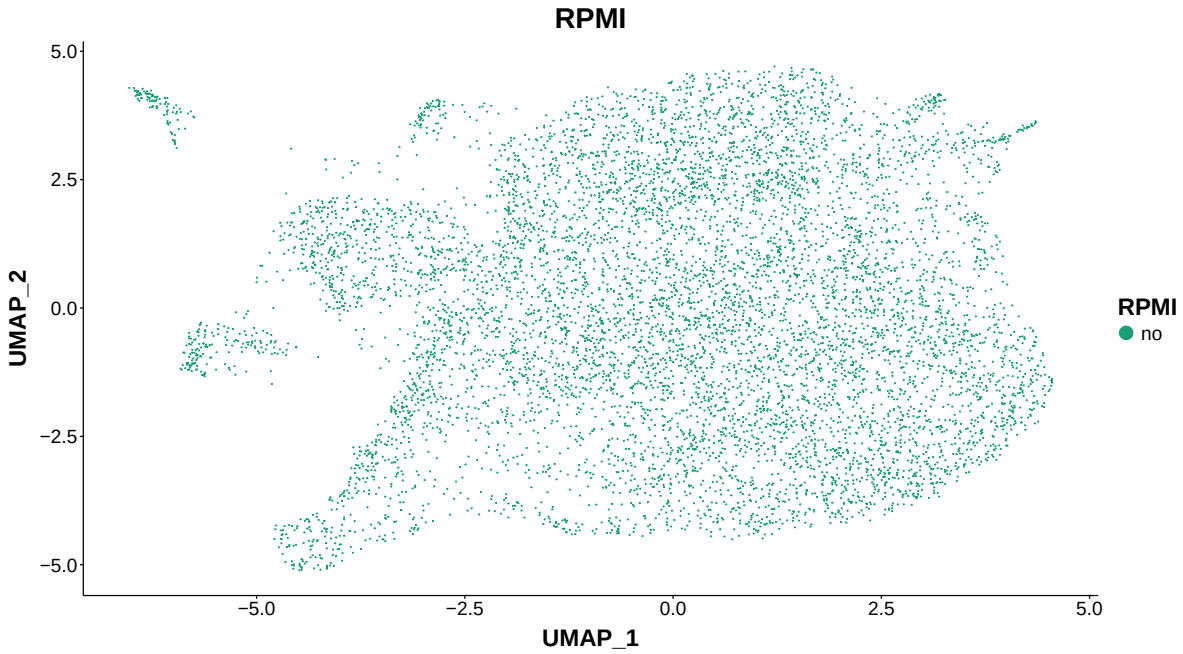

percent.mito
5.0
2.5
0.0
−2.5
−5.0
UMAP_2
−5.0
−2.5
0.0
2.5
5.0
UMAP_1
colour
0


orig.ident
5.0
2.5
0.0
−2.5
−5.0
UMAP_2
−5.0
−2.5
0.0
2.5
5.0
UMAP_1
orig.ident
DYE2547


ID
5.0
2.5
0.0
−2.5
−5.0
UMAP_2
−5.0
−2.5
0.0
2.5
5.0
UMAP_1
ID
DYE2547


cancer_type
5.0
2.5
0.0
−2.5
−5.0
UMAP_2
−5.0
−2.5
0.0
2.5
5.0
UMAP_1
cancer_type
Ewing_sarcoma






```
## [1] "Do NOT Print condition_split!"
## Returned Seurat object
```

12 Save output files

```
cat("Save Seurat object for sample", sample_name[i], "\n")
```

```
## Save Seurat object for sample DYE2547
saveRDS(seurat_obj, file = paste0(results_dir, "/", "seurat_obj.rds")) # Save Seurat object
```

13 Session Info

```
## R version 4.4.0 (2024-04-24)
## Platform: x86_64-pc-linux-gnu
## Running under: Ubuntu 22.04.4 LTS
##
## Matrix products: default
## BLAS:   /usr/lib/x86_64-linux-gnu/openblas-pthread/libblas.so.3
## LAPACK: /usr/lib/x86_64-linux-gnu/openblas-pthread/libopenblasp-r0.3.20.so;  LAPACK versi
##
## locale:
## [1] C
##
## time zone: Etc/UTC
## tzcode source: system (glibc)
##
## attached base packages:
## [1] grid      stats4    stats     graphics  grDevices utils     datasets
## [8] methods   base
##
## other attached packages:
##  [1] ggthemes_5.1.0              RColorBrewer_1.1-3
##  [3] fs_1.6.5                    scooter_0.0.0.9004
##  [5] irlba_2.3.5.1               scater_1.34.0
##  [7] scuttle_1.16.0              SingleCellExperiment_1.28.1
##  [9] SummarizedExperiment_1.36.0 Biobase_2.66.0
## [11] GenomicRanges_1.58.0        GenomeInfoDb_1.42.3
## [13] IRanges_2.40.1              S4Vectors_0.44.0
## [15] BiocGenerics_0.52.0         MatrixGenerics_1.18.1
## [17] matrixStats_1.5.0           flexmix_2.3-19
## [19] lattice_0.22-6              miQC_1.14.0
## [21] ggrepel_0.9.6               ggpmisc_0.6.1
## [23] ggpp_0.5.8-1                GGally_2.2.1
## [25] devtools_2.4.5              usethis_3.1.0
## [27] cowplot_1.1.3               optparse_1.7.5
## [29] Matrix_1.7-2                tinytex_0.54
## [31] lubridate_1.9.4             forcats_1.0.0
## [33] dplyr_1.1.4                 purrr_1.0.2
## [35] readr_2.1.5                 tidyr_1.3.1
## [37] tibble_3.2.1                ggplot2_3.5.1
## [39] tidyverse_2.0.0             stringr_1.5.1
## [41] SeuratObject_5.0.2          Seurat_4.4.0
## [43] SoupX_1.6.2                 knitr_1.49
## [45] future_1.34.0               yaml_2.3.10
##
## loaded via a namespace (and not attached):
##   [1] RcppAnnoy_0.0.22        splines_4.4.0           later_1.4.1
##   [4] polyclip_1.10-7         lifecycle_1.0.4         globals_0.16.3
##   [7] vroom_1.6.5             hdf5r_1.3.12            MASS_7.3-64
##  [10] magrittr_2.0.3          plotly_4.10.4           sass_0.4.9
##  [13] rmarkdown_2.29          jquerylib_0.1.4         remotes_2.5.0
```

```
##  [16] httpuv_1.6.15           sctransform_0.4.1       spam_2.11-1
##  [19] sp_2.2-0                sessioninfo_1.2.2       pkgbuild_1.4.6
##  [22] spatstat.sparse_3.1-0   reticulate_1.40.0       pbapply_1.7-2
##  [25] zlibbioc_1.52.0         abind_1.4-8             pkgload_1.4.0
##  [28] Rtsne_0.17              nnet_7.3-20             GenomeInfoDbData_1.2.13
##  [31] listenv_0.9.1           spatstat.utils_3.1-2    MatrixModels_0.5-3
##  [34] goftest_1.2-3           spatstat.random_3.3-2   fitdistrplus_1.2-2
##  [37] parallelly_1.42.0       DelayedArray_0.32.0     leiden_0.4.3.1
##  [40] codetools_0.2-20        getopt_1.20.4           tidyselect_1.2.1
##  [43] UCSC.utils_1.2.0        farver_2.1.2            viridis_0.6.5
##  [46] ScaledMatrix_1.14.0     spatstat.explore_3.3-4  jsonlite_1.8.9
##  [49] BiocNeighbors_2.0.1     ellipsis_0.3.2          progressr_0.15.1
##  [52] ggridges_0.5.6          survival_3.8-3          systemfonts_1.2.1
##  [55] tools_4.4.0             ragg_1.3.3              ica_1.0-3
##  [58] Rcpp_1.0.14             glue_1.8.0              SparseArray_1.6.1
##  [61] gridExtra_2.3           xfun_0.50               withr_3.0.2
##  [64] fastmap_1.2.0           SparseM_1.84-2          rsvd_1.0.5
##  [67] digest_0.6.37           timechange_0.3.0        R6_2.5.1
##  [70] mime_0.12               textshaping_1.0.0       colorspace_2.1-1
##  [73] scattermore_1.2         tensor_1.5              spatstat.data_3.1-4
##  [76] ggsci_3.2.0             generics_0.1.3          data.table_1.16.4
##  [79] FNN_1.1.4.1             S4Arrays_1.6.0          httr_1.4.7
##  [82] htmlwidgets_1.6.4       ggstats_0.8.0           uwot_0.2.2
##  [85] pkgconfig_2.0.3         gtable_0.3.6            modeltools_0.2-23
##  [88] lmtest_0.9-40           XVector_0.46.0          htmltools_0.5.8.1
##  [91] profvis_0.4.0           dotCall64_1.2           scales_1.3.0
##  [94] png_0.1-8               spatstat.univar_3.1-1   tzdb_0.4.0
##  [97] reshape2_1.4.4          nlme_3.1-167            cachem_1.1.0
## [100] zoo_1.8-12              KernSmooth_2.23-26      vipor_0.4.7
## [103] parallel_4.4.0          miniUI_0.1.1.1          pillar_1.10.1
## [106] vctrs_0.6.5             RANN_2.6.2              urlchecker_1.0.1
## [109] promises_1.3.2          BiocSingular_1.22.0     beachmat_2.22.0
## [112] xtable_1.8-4            cluster_2.1.8           beeswarm_0.4.0
## [115] evaluate_1.0.3          cli_3.6.3               compiler_4.4.0
## [118] rlang_1.1.5             crayon_1.5.3            future.apply_1.11.3
## [121] labeling_0.4.3          ggbeeswarm_0.7.2        plyr_1.8.9
## [124] stringi_1.8.4           BiocParallel_1.40.0     viridisLite_0.4.2
## [127] deldir_2.0-4            munsell_0.5.1           lazyeval_0.2.2
## [130] spatstat.geom_3.3-5     quantreg_6.00           hms_1.1.3
## [133] patchwork_1.3.0         bit64_4.6.0-1           shiny_1.10.0
## [136] ROCR_1.0-11             igraph_2.1.4            memoise_2.0.1
## [139] bslib_0.9.0             bit_4.5.0.1             polynom_1.4-1
```